中国科学院大学
《计算机组成原理(研讨课)》实验报告

姓名 王浚沣 学号 2024K8009909018 专业 计算机科学与技术

实验项目编号 1 实验名称 寄存器堆与算数逻辑单元

注 1:	撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 "-" 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
注 2:	使用 `git add` 及 `git commit` 命令将实验报告 `PDF` 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
注 3:	实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、 逻辑电路结构与仿真波形的截图及说明(比如 Verilog HDL 关键代码段 {包含注释} 及其对应的逻辑电路结构图 {自行画图, 推荐用 PPT 画逻辑结构框图后保存为 PDF, 再插入到 LaTeX. 中}、相应信号的仿真波形和信号变化的说明等)

• 寄存器堆设计

寄存器的设计整体较为简单, 根据地址位的位宽, 我们确定寄存器堆的大小, 随后用时序逻辑实现在写使能状态下数据的按地址写入即可。至于在读出数据时, 根据要求, 读出 0 号地址时默认输出为 0, 我在这里选择对读地址进行诸位或的操作, 将结果扩充至 32 位并与实际数相与, 这样就可以保证读 0 时得到 0。图 1 是寄存器堆的关键代码。图 2 则是寄存器堆的逻辑电路图。

```
        always @(posedge clk) begin
                if((wen==1)&&(waddr!=0)) begin
                        my_reg[waddr] <= wdata;
                end
        end

        assign rdata1 = my_reg[raddr1] & {32{|raddr1}};
        assign rdata2 = my_reg[raddr2] & {32{|raddr2}};

endmodule
```

图 1: 寄存器堆关键代码

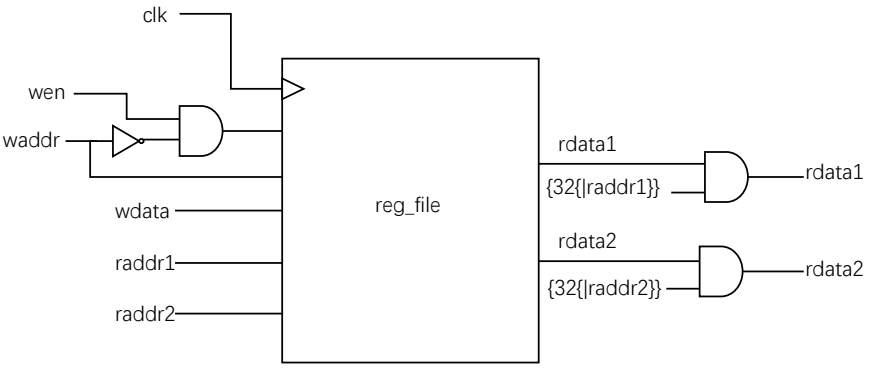


图 2: 寄存器堆逻辑电路图

观察图 3 可见,显示波形后,当时钟上升沿到达并且 wen 与 waddr 符合条件时,写地址对应的寄存器会存入相应的写数据;当读地址为 0 时,读出的数据为 0;当读地址变化为其他时,相对应的读数据也会变化为相应寄存器中写入的数据。

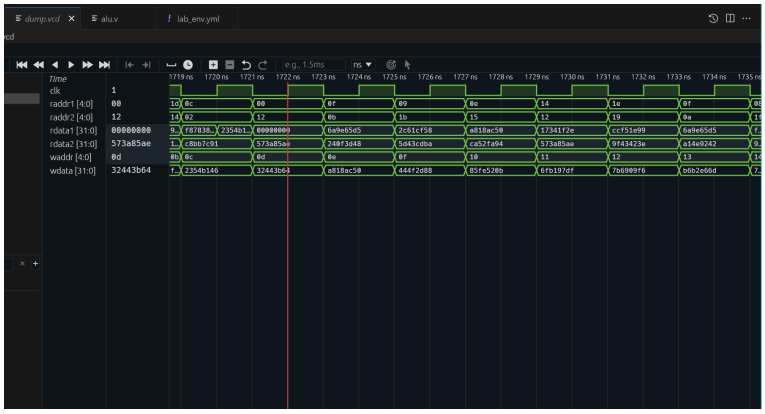


图 3: 寄存器堆仿真波形图

• **ALU 设计**

ALU 的设计分为运算逻辑与标志位逻辑。

简单的与和或运算较为简单,直接让 A 和 B 进行相应运算并输出就可以。关于加减以及比较运算,我在这里设置了一个变量 cin,cin 将根据 ALUop[3] 的数值来选择,执行加法时 cin 为 0,反之为 1。同样的,我也根据运算类型判断 B 是否需要取反操作。在进行有符号数大小判断时,我将一正一负的情况提出来单独处理,剩下的统一按减法执行,根据符号位判断比较结果,最后根据结果选择输出数。

关于符号位,ZERO 的判断是很简单的。关于 Carryout, 我在执行加法运算时扩充了一位,作为 co 输出,根据 co 的值以及执行运算的类型(加法或减法),我就可以判断 Carryout 的取值。而关于 Overflow,我通过运算类型进行分类。当执行加法时,可能的溢出形式为 A 与 B 同号且结果与 A 异号,减法则是 A 与 B 异号且结果与 A 异号,根据这个特点进行逻辑判断,即可实现符号位判断。图 4 所示的关键代码截图中便实现了我上面所说的

```
wire   [`DATA_WIDTH - 1:0]  out_and;
wire   [`DATA_WIDTH - 1:0]  out_or;
wire   [`DATA_WIDTH - 1:0]  out_add;
wire   out_slt;
wire   cin;
wire   co;
wire   [`DATA_WIDTH - 1:0] B_mux = (ALUop[2]==1) ? ~B : B;

assign cin                  = ALUop[2];
assign out_and              = A&B;
assign out_or               = A|B;
assign {co,out_add}         = A+B_mux+cin;
assign out_slt  =       (A[31] == 1 && B[31] == 0) ? 1 :
                        (A[31] == 0 && B[31] == 1) ? 0 :
                        (out_add[31] == 1) ? 1 : 0;
assign Overflow =       (ALUop == `ALUop_ADD) ? ((A[31] == B[31]) && (out_add[31] != A[31])) :
                        (ALUop == `ALUop_SUB) ? ((A[31] != B[31]) && (out_add[31] != A[31])) : 1'b0;
assign CarryOut =       (ALUop == `ALUop_ADD) ? co:
                        (ALUop == `ALUop_SUB) ? ~co:1'b0;
assign Zero     =       (Result == 32'b0);


assign Result   =       (ALUop == `ALUop_AND) ? out_and :
                        (ALUop == `ALUop_OR) ? out_or :
                        (ALUop == `ALUop_ADD||ALUop == `ALUop_SUB) ? out_add :
                        (ALUop == `ALUop_SLT) ? (out_slt ? 32'b1 : 32'b0) : 32'b0;
```

图 4: ALU 关键代码

逻辑。图 5 所示的逻辑电路图则是我代码的具体实现形式，由于部分标志位和逻辑运算较为简单，这里并没有给出相应实现。

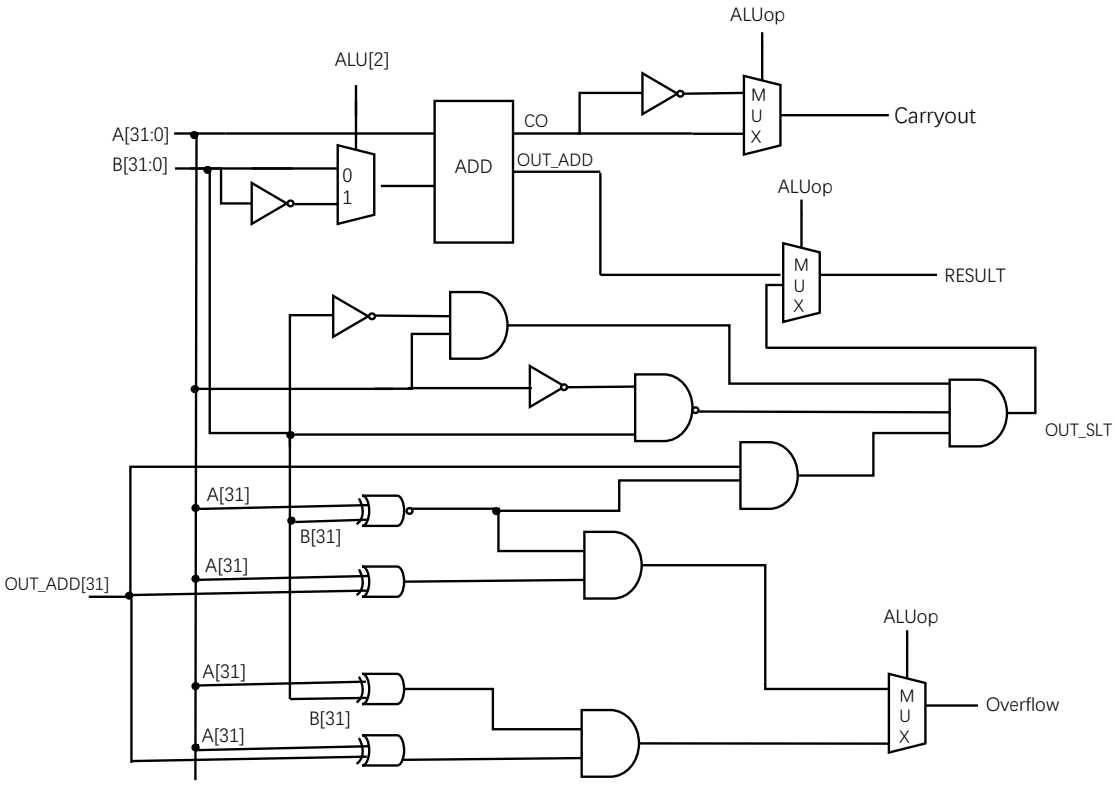
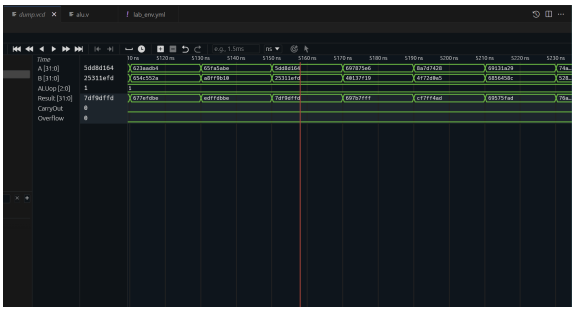
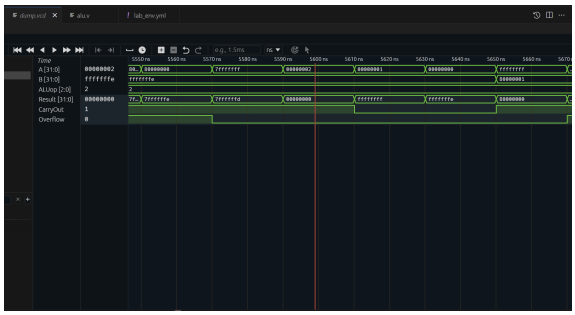


图 5: ALU 逻辑电路图

图 6 图 7 则是各运算时的波形图，当 ALUop 为不同的五种信号的时候，Result，Overflow，Carryout，Zero 会显示不同的值，并与相应的计算方式相对应。

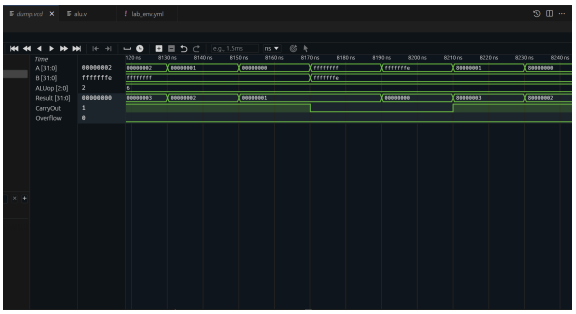


(a) 与运算

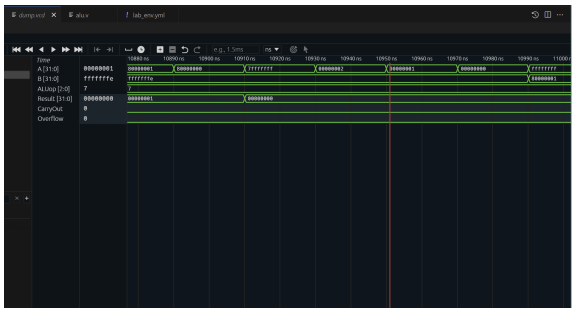


(b) 或运算

图 6: 运算波形图 1



(a) 减运算



(b) 比较运算

图 7: 运算波形图 2

二、 实验过程中遇到的问题、对问题的思考过程及解决方法(比如 RTL 代码中出现的逻辑 bug,逻辑仿真和 FPGA 调试过程中的难点等)

• **寄存器堆设计问题**。

寄存器堆的设计整体较为简单,我在设计时仅在一个地方发生了卡壳,就是在读地址为零时输出全 0。我的思路是对读地址进行逐位或,将结果重复 32 次,与原本数值相与,但由于多打了一个符号,一直通不过检测。将符号删除后,寄存器堆可以正常运行了。

助教老师检查的过程中,发现我将寄存器堆定义为 [‘DATAWIDTH - 1:0]myreg[‘DATAWIDTH - 1:0],只知道要创建一个 32 乘 32 的寄存器,却不知道为什么只能是 32 这个数,经过助教提醒后,我意识到 32 这个数字取决于 ADDRWIDTH,并对代码进行了一定更改。

```
reg    [`DATA_WIDTH - 1:0]  my_reg [(1<<(`ADDR_WIDTH))-1:0];

always @(posedge clk) begin
        if((wen==1)&&(waddr!=0)) begin
```

图 8: 寄存器堆反思修改

• **ALU 设计问题**。

ALU 设计中,符号位的输出设计和两个输入数的预处理困扰了我很久。

符号位，尤其是 Overflow 和 Carryout 两个符号的设计，我从 PPT 中的注解中获得灵感，用 A 和 B 的符号位是否相同作为判断依据，设计了 Overflow 的输出逻辑，通过扩展位以及运算类型完成了 Carryout 的设计。而关于 A 和 B 两个操作数的预处理，第一次尝试中，我错误地将 B 前再置零后取反，导致本来没有的借位信号被计算成 1。排查过后我通过 ALUop 的第二位对 B 进行预处理，并让信号进行自然的位扩展，解决了出现的问题。

助教检查时，发现我的 result 输出逻辑中存在冗余设计（out add 本身为 32 位宽信号，我在设计时由在后面添加了 [31:0] 的字样），经过提醒后已经进行修改。

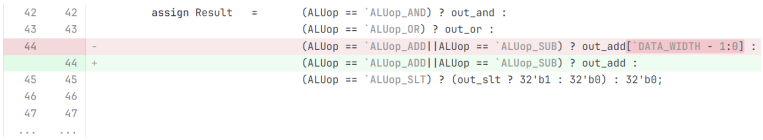


图 9: ALU 反思修改

三、 实验所耗时间

在课后，你花费了大约____8____ 小时完成此次实验。